Adaptive Feature-Space Conformal Transformation for Imbalanced-Data Learning

Gang Wu GWU@ENGINEERING.UCSB.EDU
Edward Chang ECHANG@ECE.UCSB.EDU
Department of Electrical & Computer Engineering, University of California, Santa Barbara

Abstract

When the training instances of the target class are heavily outnumbered by non-target training instances, SVMs can be ineffective in determining the class boundary. To remedy this problem, we propose the adaptive conformal transformation (ACT) algorithm. ACT considers feature-space distance and the class-imbalance ratio when it performs conformal transformation on a kernel function. Experimental results on UCI and real-world datasets show that ACT to be effective in improving class prediction accuracy.

1. Introduction

Support Vector Machines (SVMs) are a core machine learning technology. They have strong theoretical foundations and excellent empirical successes in many pattern recognition applications such as handwriting recognition [7], image retrieval [14], and text classification [10]. However, for many emerging applications, such as gene profiling and image understanding, where the training instances of the target class are significantly outnumbered by the other training instances, the class-boundary learned by SVMs can be severely skewed towards the target class. As a result, the false-negative rate can be excessively high in identifying target objects (e.g., a gene disease or a suspicious event), and hence can render the classifier ineffective. (We will present and discuss the details of this problem in Section 3.)

Several attempts have been made to improve class-prediction accuracy of SVMs [1, 2, 4, 11, 12, 17]. Given the class prediction function of SVMs,

$$sgn(f(\mathbf{x}) = \sum_{i=1}^{n} y_i \alpha_i K(\mathbf{x}, \mathbf{x}_i) + b), \quad (1)$$

we can identify four parameters that affect the decision outcome: b, α_i, γ, and K. Our empirical study shows that the only effective method for improving SVMs is through modifying the kernel function K. As indicated by Amari and Wu [1, 17], by conformally spreading the area around the class-boundary outward on the Riemannian manifold S where all mapped data are located in feature space F, we can adapt K locally to data distribution to improve class-prediction accuracy.

In this paper, we propose an adaptive conformal transformation (ACT) algorithm. ACT improves upon Amari and Wu's method for tackling the imbalanced-training-classes problem in two respects.

1. We conduct the transformation based on the spatial distribution of the support vectors in feature space F, instead of in input space I [17]. Using feature-space distance to conduct conformation transformation takes advantage of the new information learned by SVMs in every iteration, whereas input-space distance remains unchanged.

2. We adaptively control the transformation based on the skew of the class-boundary. This transformation gives the neighborhood of minority support vectors a higher spatial resolution, and hence achieves better separation between the classes.

Our experimental results on both UCI and real-world image datasets show ACT to be very effective in correcting the skewed boundary.

The rest of this paper is organized as follows. Section 2 discusses related work. Section 3 uses a 2-D example to explain the problem caused by imbalanced training data for SVMs. In Section 4 we describe the ACT algorithm for tackling the imbalanced training-data problem. Section 5 presents the setup and the results of our empirical studies. We offer our concluding remarks in Section 6, as well as suggestions for further studies.

2. Related Work

Researchers have modified the learning algorithm of SVMs for remedying the imbalanced training-data problem. Let us revisit the class prediction function in Eq. 1. We can identify four parameters that affect the decision outcome: b, α_i, γ, and K. Related algorithmic work can be summarized by changing these parameters.

- b. The b parameter is the intercept. Some researchers change b to shift the decision boundary. However, this naive boundary-movement method is a post-processing method. Intuitively, we can see that changing b trades a higher false positive count for a lower false negative one.
- α_i. The α_i determines the magnitude of influence of training instance $\mathbf{x}_i$. According to Eq. 1, the larger the α_i, the larger the influence of x_i in class prediction. Based on this idea, Veropoulos et al. [16] and Lin et al. [12] use different penalty constants for different classes to tune the α_i. It turns out that this biased-penalty method does not help SVMs as much as expected. According to the KKT conditions, the value of α_i has three ranges [8]:

$$\alpha_i = 0 \Rightarrow y_i f(x_i) \geq 1 \quad \text{and} \quad \xi_i = 0 \tag{2}$$

$$0 < \alpha_i < C \Rightarrow y_i f(x_i) = 1 \quad \text{and} \quad \xi_i = 0 \tag{3}$$

$$\alpha_i = C \Rightarrow y_i f(x_i) \leq 1 \quad \text{and} \quad \xi_i \geq 0. \tag{4}$$

  In addition, α_i's are constrained by

$$\sum_{i=1}^{n} \alpha_i y_i = 0. \tag{5}$$

  We can see that C imposes only an upper bound on α_i, not a lower bound. Increasing C does not necessarily affect α_i. Moreover, the constraint in Eq. 5 imposes an equal total influence from the positive and negative support vectors. The increases in some α_i at the positive side will inadvertently increase some α_i at the negative side to satisfy the constraint. These constraints can make the increase of C^+ on minority instances ineffective.
- γ. The γ parameter determines the width of an RBF function. For instance, when a Gaussian function is employed as the kernel function, the γ parameter decides the width ($\gamma = 1/\sigma^2$) of the Gaussian function. The smaller the γ, the larger the σ, and hence the larger the influence of a support vector in its neighborhood. To tackle the imbalanced training problem, one can assign different γ values for different classes. This γ scaling method does not show much improvement on learning imbalanced datasets, since it assigns a different γ for each class, not for each support vector.
- K. The kernel function affects the distance computation between a support vector $\mathbf{x}_i$ and a testing instance $\mathbf{x}$. (We assume that K is an RBF function in this paper.) Based on this idea, Amari and Wu [1, 17] propose to change the kernel using a conformal transformation method. In this paper, we further Amari and Wu's work to deal with imbalance-data training.

An entirely orthogonal approach is data-processing, which subsamples the majority class or oversamples the minority class in the hope of reducing the skew of the training dataset. One-sided selection by Kubat [11] is a representative subsampling approach, which removes noisy, borderline, and redundant majority training instances. However, these steps typically can remove only a small fraction of the majority instances, and may not be very helpful in a scenario with a ratio of more than 100 : 1, which is becoming common in many emerging applications. Multi-classifier training proposed by Chan and Stolfo [4], and Bagging proposed by Breiman [2], are two other subsampling methods. These methods do not deal with noisy and borderline data directly, but use a large ensemble of subclassifiers to reduce prediction variance.

Oversampling [5] is the opposite of the downsampling approach. It duplicates or interpolates minority instances in the hope of reducing the imbalance. The oversampling approach can be considered as a "phantom-transduction" method. It assumes the neighborhood of a positive instance to be still positive, and the instances between two positive instances positive. However, assumptions like these can be data-dependent and unreliable. We believe that employing an effective data-processing approach can complement an algorithmic approach. Our focus in this paper is on developing an algorithmic approach.

3. Boundary Bias and SVMs

A subtle but severe problem that an SVM classifier faces is the skewed class boundary caused by imbalanced training data. To illustrate this problem, Figure 1 depicts a 2D checkerboard example. The checkerboard divides a 200×200 square into four quadrants. The top-left and bottom-right quadrants contain negative (majority) instances while the top-right and bottom-left quadrants are occupied by positive (minority) instances. The lines between the classes are the "ideal" boundary that separates the two classes. In the rest of the paper, we will use *positive* when referring to minority instances, and *negative* when referring to majority instances.

Figure 2 exhibits the boundary distortion between the two left quadrants in Figure 1 under two different negative/positive training-data ratios, where a black dot with a circle represents a support vector, and its radius represents the weight value α_i of the support vector. The bigger the circle, the larger the α_i. Figure 2(a) shows the SVM class boundary when the ratio of the number of negative instances (in the quadrant above) to

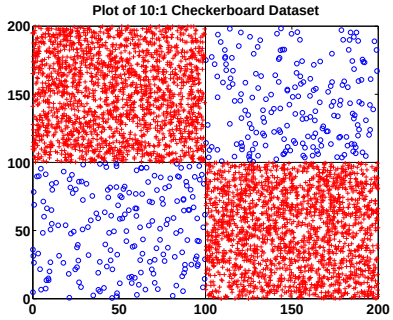
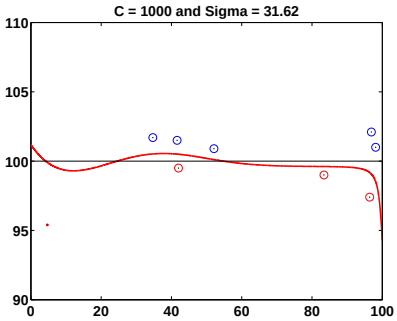
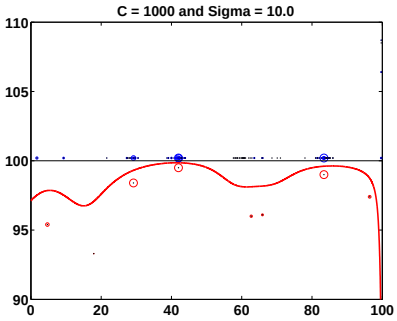


Figure 1. Checkerboard Experiment.

the number of positive instances (in the quadrant below) is $10:1$. Figure 2(b) shows the boundary when the ratio increases to $10,000:1$. The boundary in Figure 2(b) is much more skewed towards the positive quadrant than the boundary in Figure 2(a), and hence causes a higher incidence of false negatives.



(a) 10:1



(b) 10,000:1

Figure 2. Boundaries of Different Ratios.

To explain the causes of the boundary-skew phenomenon, Figure 3 plots the negative-to-positive support-vector ratios (nn/np). The x-axis of the figure depicts the ratios of negative-to-positive training instances, and the y-axis the ratios of negative-to-positive support vectors. The y-axis shows that the support-vector ratio grows as the imbalance between the classes grows. Using the SVM properties and Figure 3, we observe two potential causes for the boundary skew.

1. *The imbalanced training-data ratio.* At the minority

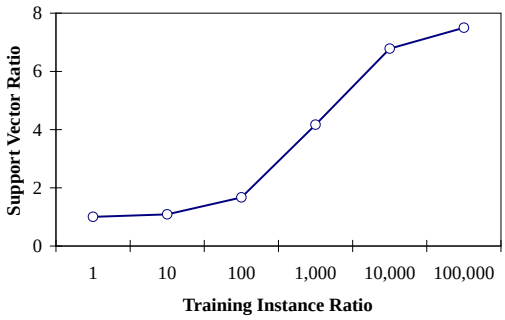


Figure 3. Support Vectors Ratios.

side of the boundary, the positive training data may not always reside as close to the "ideal boundary" as the negative training data do. This can be explained by a simple example. Suppose we randomly draw n numbers based on a uniform distribution between 1 and 100. The larger the value of n, the higher the chance that we draw a number close to 100, though the expected mean of the draws is invariant of the values of n. Thus, the low presence of the positive training instances makes them appear farther from the "ideal boundary" than the negative training instances.

2. *The imbalanced support-vector ratio.* When inspecting the boundary data in Figure 2, we find the α_i values of the minority class (the positive class) tend to be much larger than those of the majority class, while the number of positive support vectors is substantially smaller. This phenomenon agrees with the constraint presented in Eq. 5. As a consequence, the nearest neighborhood of a testing point, especially when it is near the boundary, is likely to be dominated by negative support vectors, and hence the decision function (Eq. 1) is more likely to classify a boundary point negative.

4. Adaptive Conformal Transformation

In this section, we first explain the geometry of the feature space, from which we obtain information to perform conformal transformation. We then propose the Adaptive Conformal Transformation (ACT) algorithm. We close by pointing out the differences between ACT and the prior work [1, 17].

4.1. Geometry of Feature Space F

A support vector machine can use the *kernel mapping* [15, 8] to map the data from a Euclidean input space I to a high-dimensional Hilbert feature space F, in which a classification or regression problem becomes linear. In general, the surface S has the same dimensionality as that of the input space I [3]. The shape of S is determined by the associated mapping Φ. If we assume that the map Φ has all continuous derivatives,

as in the case of an RBF function, the surface S in F is smooth, and thus can be considered as a Riemannian manifold [3], which enables us to define a Riemannian metric for S.

The relationship between I and S can be imagined from a simple Earth example. The local spot where we stand is perceived as flat, but actually the Earth itself is a globe. Figure 4 illustrates another example. Surface 1 shows that a $\Re^2$ input space I is mapped to an irregular surface in feature space F under one mapping function Φ_1. In this case, for one point P in this irregular surface, its neighborhood is locally $\Re^2$. One can also choose a different Φ_2 to map $\Re^2$ to the surface of a globe in F, where the neighborhood of one point P is still locally $\Re^2$.

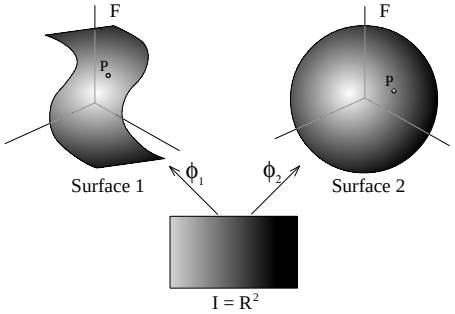


Figure 4. A Riemannian-Manifold Example.

4.1.1. Riemannian Metric

Ordinarily, we use two schemes to measure the distance between two points on S, as schematically illustrated in Figure 5. The first one considers the distance between two points along a straight line in F, which is the so-called Euclidean distance; the second one measures the distance between two points along a path on S by integration. This distance, called the Riemannian distance [3], is computed by a metric induced on S. This metric is thus called the Riemannian metric, denoted as g_{ij} in this paper.

A Riemannian metric tells us how to compute the distance between any two points on S. The components of a Riemannian metric can be viewed as multiplication factors which are placed before the differential displacements dx_i in I to compute the distance ds of an element dz in F in a generalized Pythagorean theorem,

$$ds^2 = \sum_{i,j} g_{ij} dx_i dx_j. \tag{6}$$

The Euclidean distance can be also regarded as some format of the Riemannian distance, where g_{ij} is a discrete delta function δ_{ij}[1],

$$ds^2 = \sum_i {dx_i}^2. \tag{7}$$

[1] A discrete delta function δ_{ij} equals 1 only when $i = j$, else equals 0.

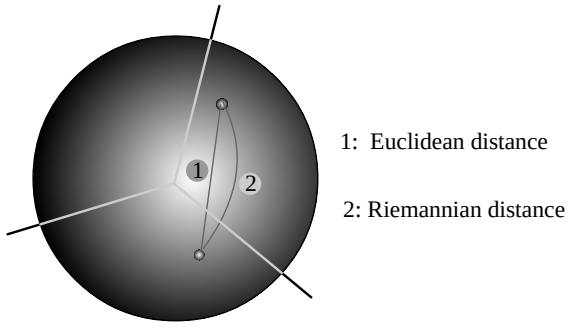


Figure 5. Two Distance Measures in a 3-D Feature Space.

4.1.2. From Kernel to Riemannian Metric

Although the kernel mapping Φ is unknown, one can still calculate the Riemannian metric induced on S by playing the kernel trick.

Let $\mathbf{z}$ denote the mapped pattern of $\mathbf{x}$ in F. We have $\mathbf{z} = \Phi(\mathbf{x})$. Under this mapping function, a small vector $d\mathbf{x}$ is mapped to

$$d\mathbf{z} = \Phi(\mathbf{x} + d\mathbf{x}) - \Phi(\mathbf{x}). \tag{8}$$

Then, the Riemannian distance ds is defined as

$$\begin{aligned} ds^2 &= \|d\mathbf{z}\|^2 \\ &= \|\Phi(\mathbf{x} + d\mathbf{x}) - \Phi(\mathbf{x})\|^2 \\ &= K(\mathbf{x} + d\mathbf{x}, \mathbf{x} + d\mathbf{x}) - 2K(\mathbf{x}, \mathbf{x} + d\mathbf{x})K(\mathbf{x}, \mathbf{x}) \\ &= \sum_{i,j} \left(\frac{1}{2} \frac{\partial^2 K(\mathbf{x}, \mathbf{x})}{\partial x_i \partial x_j} - \left(\frac{\partial^2 K(\mathbf{x}, \mathbf{x}')}{\partial x'_i \partial x'_j} \right)_{\mathbf{x}=\mathbf{x}'} \right) dx_i dx_j. \end{aligned}$$

From Eq. 6, we can see that the Riemannian metric induced on S can be defined as

$$g_{ij}(\mathbf{x}) = \frac{1}{2} \frac{\partial^2 K(\mathbf{x}, \mathbf{x})}{\partial x_i \partial x_j} - \left(\frac{\partial^2 K(\mathbf{x}, \mathbf{x}')}{\partial x'_i \partial x'_j} \right)_{\mathbf{x}=\mathbf{x}'}. \tag{9}$$

It shows how a local area in I is magnified in F under the mapping $\Phi(\mathbf{x})$ [3].

4.2. Adaptive Conformal Transformation

A conformal transformation, also called a conformal mapping, is a transformation T which takes the elements $X \in D$ to elements $Y \in T(D)$ while preserving the local angles between the elements after mapping, where D is a domain in which the elements X reside [6]. Figure 6 shows a example using two different conformal transformations on a grid structure, where $\mathbf{z}$ is a complex variable. We can see that the local grid structure is almost invariant after such a transformation (that is, "*conformal*"). Usually, an analytic function[2] is conformal at any point where it has a nonzero derivative. In this paper, we use the term conformal function. Some

[2] An analytic function is a complex function which is complex differentiable at every point in a region R. Please refer to some mathematical books for the details.

commonly used conformal functions are represented as $\mathbf{z}^2$, $e^{-\mathbf{z}}$, and $e^{-\mathbf{z}^2}$.

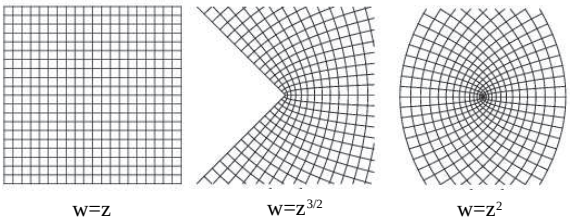


Figure 6. A Conformal Transformation Example.

In section 4.1, we noted that a Riemannian metric $g_{ij}(\mathbf{x})$ induced on S shows how a local area around $\mathbf{x}$ in I is magnified in F under the mapping Φ. The idea of conformal transformation in SVMs is to enlarge the magnification factor $g_{ij}(\mathbf{x})$ around the boundary but reduce it around other points by modifying the related function K, according to Eq. 9. This can be implemented by introducing a conformal transformation of kernel [1, 17]

$$\tilde{K}(\mathbf{x},\mathbf{x}') = D(\mathbf{x})D(\mathbf{x}')K(\mathbf{x},\mathbf{x}'), \tag{10}$$

where $D(x)$ is a properly defined positive function.

4.2.1. Selection of $D(\mathbf{x})$

Substituting Eq. 10 into Eq. 9 with the form of a new kernel $\tilde{K}(\mathbf{x},\mathbf{x}')$, the new Riemannian metric $\tilde{g}_{ij}(\mathbf{x})$ is

$$\tilde{g}_{ij}(\mathbf{x}) = D(\mathbf{x})^2 g_{ij}(\mathbf{x}) + D'_i(\mathbf{x})D'_j(\mathbf{x}) + 2D'_i(\mathbf{x})D(\mathbf{x})K'_i(\mathbf{x},\mathbf{x}), \tag{11}$$

where $K'_i(\mathbf{x},\mathbf{x}) = \partial K(\mathbf{x},\mathbf{x}')/\partial x_i|_{\mathbf{x}'=\mathbf{x}}$ and $D'_i(\mathbf{x}) = \partial D(\mathbf{x})/\partial x_i$. To further increase the margin of SVMs in F, $D(\mathbf{x})$ should be chosen in a way such that $\tilde{g}_{ij}(\mathbf{x})$ has greater values around the decision boundary. In addition to dealing with the skew of the class-boundary, $\tilde{g}_{ij}(\mathbf{x})$ is greater especially for the boundary area close to the minority class. An RBF distance function is a good choice for $D(\mathbf{x})$.

In practice, since it is unknown exactly where the optimal boundary is located, the positions of support vectors are used to approximate the class-boundary [1]. Conformal transformation then increases the metric in the neighborhood of the support vectors. The degree of such magnification depends on the selection of conformal function $D(\mathbf{x})$.

Suppose we choose the conformal function $D(\mathbf{x})$ as $\exp(-\frac{\|\mathbf{x}-\mathbf{x}_k\|^n}{\tau_k^2})$, and the initial kernel $K(\mathbf{x},\mathbf{x}')$ as $\exp(-\gamma\|\mathbf{x}-\mathbf{x}'\|^m)$. Due to Eq. 11, the new Riemannian metric in the neighborhood of the support vector $\mathbf{x}_k$ is given by

$$\begin{aligned}\tilde{g}_{ij}(\mathbf{x}) &= D(\mathbf{x})^2 g_{ij}(\mathbf{x}) && (12)\\ &\quad + D(\mathbf{x})^2 \frac{n^2}{\tau_k^4}(x_i - x_{ki})^{n-1}(x_j - x_{kj})^{n-1} && (13)\\ &\quad + D(\mathbf{x})^2 \frac{\gamma n}{\tau_k^2}(x_i - x_{ki})^{n-1}\delta(1), && (14)\end{aligned}$$

where $\delta(1)$ means the last term exits only when the initial kernel $K(\mathbf{x},\mathbf{x}')$ is L_1-norm RBF distance function.

From Eq. 14, we can see the new metric $\tilde{g}_{ij}(\mathbf{x})$ is associated with the norm number n and the parameter τ_k of $D(\mathbf{x})$. Since $x_i - x_{ki}$ is very small in the $\mathbf{x}_k$ neighborhood, $\tilde{g}_{ij}(\mathbf{x})$ would decrease when n increases. We therefore prefer $D(\mathbf{x})$ with a smaller norm, such as L_1 or L_2. In the remainder of this section, for simplification, we will discuss only the case when both $K(\mathbf{x},\mathbf{x}')$ and $D(\mathbf{x})$ are the L_1-norm RBF function.

4.2.2. Selection of τ_k for $D(\mathbf{x})$

Let $D(\mathbf{x})$ be a L_1-norm RBF function. Eq. 14 becomes

$$\tilde{g}_{ij}(\mathbf{x}) = D(\mathbf{x})^2 g_{ij}(\mathbf{x}) + D(\mathbf{x})^2 \frac{1}{\tau_k^4}. \tag{15}$$

From Eq. 15, we can see that in the neighborhood of the support vector $\mathbf{x}_k$, $\tilde{g}_{ij}(\mathbf{x})$ is inversely proportional to τ_k. To reflect the spatial distribution of the support vectors in F, we hope the τ_k would be smaller for the area in feature space F where the support vectors are dense, so as to get a greater magnification metric $\tilde{g}_{ij}(\mathbf{x})$; otherwise τ_k will be greater for the area in F where the support vectors are scarce. In this case, $D(\mathbf{x})$ is

$$D(\mathbf{x}) = \sum_{k\in SV} \exp(-\frac{|\mathbf{x}-\mathbf{x}_k|}{\tau_k^2}), \tag{16}$$

where τ_k can be chosen as

$$\tau_k^2 = \mathrm{AVG}_{i\in\{\|\Phi(\mathbf{x}_i)-\Phi(\mathbf{x}_k)\|^2<M,\ y_i\neq y_k\}}\left(\|\Phi(\mathbf{x}_i)-\Phi(\mathbf{x}_k)\|^2\right). \tag{17}$$

In Eq. 17, the average comprises all support vectors falling into its neighborhood with the radius of M but having different class labels with $\Phi(\mathbf{x}_i)$. Here, M is the margin of SVMs. Choosing τ_k^2 in this way takes into consideration the spatial distribution of the support vectors in F. In addition, if there are support vectors falling into the margin, this technique can be applied to reshape the support vector hyperplane in F. In this case, the support vector hyperplane is not regularly shaped. We hope to provide the support vectors in the margin with larger magnification so as to spread them toward their class area[3]. Since these support vectors are closest to the opposite class area, according to Eq. 15 and Eq. 17, they can achieve higher magnification by calculating the distance with support vectors that belong to the other classes.

Though the mapping function Φ is unknown, we can play the kernel trick to calculate the distance in F:

$$\|\Phi(\mathbf{x}_i)-\Phi(\mathbf{x}_k)\|^2 = K(\mathbf{x}_i,\mathbf{x}_i) + K(\mathbf{x}_k,\mathbf{x}_k) - 2K(\mathbf{x}_i,\mathbf{x}_k). \tag{18}$$

[3]This method would fail if there were support vectors falling not in the margin, but in the area of the other class. In this case, they are outliers, which then could be removed from the training dataset.

Substituting Eq. 18 into Eq. 17, we then can calculate a τ_k for each support vector, which can adaptively reflect the spatial distribution of the support vector in the feature space, not in the input space.

4.2.3. Tuning τ_k for imbalanced classes

When the training dataset is very imbalanced, the class-boundary would be skewed toward the minority class in input space I. In this case, in I, the minority support vectors are located more closely to the class-boundary than the majority ones. Due to Eq. 15, we hope the new metric $\tilde{g}_{ij}(\mathbf{x})$ would further magnify the area far away from a minority support vector $\mathbf{x}_i$ so that the boundary imbalance could be alleviated. Our algorithm thus assigns a coefficient for the τ_k in Eq. 17 to reflect the boundary skew in $D(\mathbf{x})$. The new $\tilde{\tau}_k$ is as follows:

If $\mathbf{x}_k$ is a minority support vector,

$$\tilde{\tau}_k^2 = \eta_p \tau_k^2. \tag{19}$$

Else

$$\tilde{\tau}_k^2 = \eta_n \tau_k^2. \tag{20}$$

Examining Eq. 15 and Eq. 16, we can see that $D(\mathbf{x})$ is a monotonously increasing function of τ_k. To increase the metric $\tilde{g_{ij}}(\mathbf{x})$ in an area which is not very close to the support vector $\mathbf{x}_k$, it would be better to choose a larger η_p for the τ_k^2 of a minority support vector. For a majority support vector, we can maintain its coefficient η_n as 1 or use a smaller value, so as to minimize influence on the class-boundary. We empirically demonstrate that η_p and η_n are proportional to the skew of support vectors, or η_p as $O(\frac{|SV+|}{|SV-|})$, and η_n as $O(\frac{|SV-|}{|SV+|})$, where $|SV+|$ and $|SV-|$ denote the number of minority and majority support vectors, respectively.

Figure 7 summarizes the ACT algorithm. We apply ACT on the training dataset X_{train} until the testing accuracy on X_{test} cannot be further improved. In each iteration, ACT adaptively calculates τ_k^2 for each support vector (step 9), based on the distribution of support vectors in feature space F. ACT scales the τ_k^2 according to the negative-to-positive support-vector ratio (steps 10 to 13). Finally, ACT updates the kernel and performs retraining on X_{train} (steps 14 to 16).

4.3. Comparison with Traditional Conformal Transformation

Compared to the traditional conformal transformation [1, 17], our ACT algorithm has two differences:

1. The selection of τ_k is dynamic in the tradition conformal transformation, but not adaptive, since the distance between two support vectors is calculated in input space I, not in feature space F. Calculating τ_k in I does not reflect the spatial distribution of the Riemannian manifold S. Furthermore, the input-space distance is unchanged throughout the transformation iterations. In ACT, the τ_k for the conformal function is calculated based on its distribution in F.

2. The traditional conformal transformation does not address the problem of boundary bias in the imbalanced training data scenario. In such a case, the decision boundary locates much closer to the minority class than to the majority one. We assign a larger metric $g_{ij}(\mathbf{x})$ for the boundary area close to the minority class than for that close to the majority class, so as to reduce the skew of the class-boundary.

Input:
X_{train}, X_{test}, θ, K, γ;
Output:
$\mathcal{C}$; /* *output classifier* */
Variables:
SV; /* *support vector set* */
M; /* *SVM margin* */
s; /* *a support vector* */
s.τ; /* *parameter of S* */
s.y; /* *class label of S* */
Function Calls:
SVMTrain(X_{train}, K, γ); /* *train SVM classifier* $\mathcal{C}$ */
SVMClassify($X_{test}, \mathcal{C}$); /* *classify* X_{test} *by* $\mathcal{C}$ */
ExtractSV($\mathcal{C}$); /* *obtain* **SV** *from* $\mathcal{C}$ */
ComputeMargin($\mathcal{C}$); /* *compute margin* */

Begin
1) $\mathcal{C} \leftarrow$ SVMTrain(X_{train}, K, γ);
2) $\varepsilon_{old} \leftarrow \infty$;
3) $\varepsilon_{new} \leftarrow$ SVMClassify($X_{test}, \mathcal{C}$);
4) **while** ($\varepsilon_{new} - \varepsilon_{old} > \theta$) {
5) **SV**$\leftarrow$ExtractSV($\mathcal{C}$);
6) $M \leftarrow$ComputeMargin($\mathcal{C}$);
7) $\eta_p \leftarrow O(\frac{|SV^-|}{|SV^+|})$, $\eta_n \leftarrow O(\frac{|SV^+|}{|SV^-|})$;
8) **for** each $\mathbf{s} \in$ **SV** {
9) $\mathbf{s}.\tau \leftarrow sqrt(\text{AVG}_{i \in \{\|\Phi(\mathbf{s}_i)-\Phi(\mathbf{s})\|^2 < M,\ \mathbf{s}_i.y \neq \mathbf{s}.y\}} \left(\|\Phi(\mathbf{s}_i) - \Phi(\mathbf{s})\|^2\right))$
10) **if** $\mathbf{s} \in \mathbf{SV}^+$ **then** /* *a minority support vector* */
11) $\mathbf{s}.\tau \leftarrow \sqrt{\eta_p} \times \mathbf{s}.\tau$;
12) **else** /* *a majority* */
13) $\mathbf{s}.\tau \leftarrow \sqrt{\eta_n} \times \mathbf{s}.\tau$; }
14) $D(\mathbf{x}) = \sum_{\mathbf{s} \in SV} \exp\left(-\frac{\|\mathbf{x}-\mathbf{s}\|^2}{\mathbf{s}.\tau^2}\right)$
15) $K \leftarrow D(\mathbf{x}) \times D(\mathbf{x}') \times K$;
16) $\mathcal{C} \leftarrow$ SVMTrain(X_{train}, K, γ);
17) $\varepsilon_{old} \leftarrow \varepsilon_{new}$;
18) $\varepsilon_{new} \leftarrow$ SVMClassify($X_{test}, \mathcal{C}$); }
19) **return** $\mathcal{C}$;
End

Figure 7. Algorithm ACT.

5. Experimental Results

Let CT_I denote Wu and Amari's conformal transformation algorithm based on input-space distance, ACT_I denote the ACT that uses input-space distance to transform $D(\mathbf{x})$, and ACT_F denote the ACT using feature-space distance. Our empirical study examined the effect of algorithm ACT in two aspects.

Dataset	attributes	positives	negatives
seg1	19	30	180
g7	10	29	185
euth1	24	238	1762
car3	6	69	1659
yeast5	8	51	1433
ab19	8	32	4145

Table 1. Six UCI Benchmark Datasets.

Dataset	C4.5	SVMs	CT_I	ACT_I	ACT_F
seg1	97.9	98.1	96.2	98.1	**98.1**
g7	91.3	89.9	85.6	93.7	**93.7**
euth1	93.5	92.6	94.1	94.3	**94.5**
car3	89.3	99.0	99.1	99.1	**99.9**
yeast5	**84.7**	59.1	71.8	75.6	78.5
ab19	0.0	0.0	44.2	50.5	**51.8**

Table 2. UCI-dataset Prediction Accuracy.

1. *CT_I versus ACT_I.* We evaluated the marginal effect of using adaptive τ_k for shaping the conformal kernel $D(\mathbf{x})$. Both methods shape $D(\mathbf{x})$ in I, and ACT_I in addition takes the imbalance-class ratio into consideration.

2. *ACT_I versus ACT_F.* We evaluated the marginal effect of using feature-space distance for shaping the conformal kernel $D(\mathbf{x})$ compared to using input-space distance.

For each experiment, we selected an RBF function (Gaussian or Laplacian) for $K(\mathbf{x}, \mathbf{x}')$ that worked the best on the dataset. We employed the best parameter settings (C and γ) obtained through employing cross-validation. For each experiment, we performed five runs of stratified 7-fold cross-validation, computed average class-prediction accuracy, and compared the results. For our experiments, we used six UCI datasets and a 20K, 116-category image dataset, which are described as follows:

Six UCI datasets The six UCI datasets we experimented with are, *abalone* (abalone19), *car* (car3), *segmentation* (seg1), *yeast* (yeast5), *glass* (g7), and *euthyroid* (euthy1). The number in the parenthesis depicts the target class we chose. Table 1 shows the characteristics of these six datasets organized according to their negative-to-positive training-instance ratios. The top three datasets (seg1, g7, and euth1) are not-too-imbalanced. The middle two (car3 and yeast5) are mildly imbalanced. The bottom dataset (ab19) is imbalanced (the ratio is 130).

20K-image dataset The image dataset contains 20K, 116 categories of images collected from the Corel Image CDs[4]. Each image is represented by a vector of 144 dimensions including color, texture, and shape features [14]. To perform class prediction, we employed the one-per-class (OPC) ensemble [9], which trains 116 classifiers, each of which predicts the class membership for one class. The class prediction on a testing instance is decided by voting among the 116 classifiers. Table 4 presents results of 12 selected categories also organized into three groups according to the imbalance ratio (the imbalance ratios are listed on the second column).

[4]We exclude from our testbed categories that are not possible to classify automatically, such as "industry", "Rome", and "Boston". (E.g., the Boston category contains various subjects, e.g., architectures, landscapes, and people, of Boston.)

5.1. Results on UCI Benchmark Datasets

We first report the experimental results with the six UCI datasets in Tables 2 and 3. In addition to conducting experiments with SVMs, CT_I, ACT_I, and ACT_F, we also implemented and tested one other popular statistical method C4.5 [13]. We used the L_2-norm RBF function for $D(\mathbf{x})$, the same function used by [17]. We report in Table 2 using the Kubat's g-means metric defined as $\sqrt{a^+ \cdot a^-}$, where a^+ and a^- are positive (the target class) and negative testing accuracy, respectively [11]. The table shows that ACT_I outperforms CT_I by a significant margin, especially when the imbalance ratio is large (ab19). ACT_F performs about the same or slightly better than ACT_I. ACT_F achieves the best accuracy in five of the six datasets. When the data is very imbalanced (the last row of Table 2), only ACT_F works reasonably effective. Table 3 presents a^+ and a^- separately. ACT_T achieves the best a^+ while maintaining a good a^-. (We mark the best g-means and a^+ in bold font in Table 2 and Table 3, respectively.)

5.2. Results on $20k$ Image Dataset

The image dataset is more imbalanced than the UCI datasets. We first set aside $4K$ images to be used as testing data; the remaining $16K$ images were used for training. We compared four schemes: SVMs, CT_I, ACT_I, and ACT_F. Notice that in this experiment, we use the L_1-norm RBF function for $D(\mathbf{x})$, since the L_1-norm RBF works the best for the image dataset.

Table 4 presents the classification accuracy for twelve representative categories out of 116 ones, organized by their imbalance ratios. First, there was no significant increase in the average classification accuracy for the CT_I scheme over that of SVMs. Second, compared to SVMs, the ACT_I scheme improves prediction accuracy by 3.8%, 5.0%, and 9.1% on the three subgroups datasets, respectively. Finally, ACT_F's improvement is even more significant: it outperforms by 7.6%, 4.9%, and 13.4%, respectively. In addition, ACT_F achieves the best classification accuracy for eleven out of twelve categories

Data		C4.5	SVMs	CT_I	ACT_I	ACT_F
seg1	a^+	92.9	96.4	92.9	96.4	**96.4**
	a^-	99.4	100.0	100.0	100.0	100.0
g7	a^+	85.7	82.1	75.0	89.3	**89.3**
	a^-	97.8	98.9	98.4	98.9	98.9
euth1	a^+	88.2	87.4	90.7	91.5	**92.0**
	a^-	99.0	98.3	97.7	97.3	97.1
car3	a^+	81.0	98.4	98.4	98.4	**100.0**
	a^-	99.2	99.7	99.8	99.8	99.8
yeast5	a^+	**73.5**	36.7	53.1	61.2	63.3
	a^-	98.7	98.5	97.6	96.4	97.6
ab19	a^+	0.0	0.0	25.0	28.6	**28.6**
	a^-	99.6	99.5	94.0	90.1	94.0

Table 3. UCI-dataset Prediction Accuracy.

Category	Ratio	SVMs	CT_I	ACT_I	ACT_F
Mountain	34 : 1	24.8	23.9	25.7	**33.3**
Snow	37 : 1	46.4	49.5	51.2	**54.6**
Desert	39 : 1	33.7	35.9	37.1	**39.1**
Dog	44 : 1	32.9	37.8	38.4	**41.5**
Woman	54 : 1	27.9	30.9	32.2	**35.3**
Church	66 : 1	21.8	21.8	**23.6**	20.0
Leaf	80 : 1	26.1	30.4	33.6	**32.6**
Lizard	101 : 1	13.9	16.0	20.0	**22.2**
Parrot	263 : 1	7.1	7.1	14.3	**14.3**
Horse	264 : 1	14.3	21.4	28.6	**28.6**
Leopard	283 : 1	7.7	0.0	14.3	**23.1**
Shark	1232 : 1	0.0	0.0	8.4	**16.6**

Table 4. Image-dataset Prediction Accuracy.

among all schemes(marked by bold font). ACT is especially effective when training classes are imbalanced.

6. Conclusion and Future Work

We have proposed an adaptive feature-space conformal transformation (ACT) algorithm, for tackling the imbalanced training-data challenge. ACT adaptively changes the shape of Riemannian manifold S where all mapped training instances reside, based on the spatial distribution of all support vectors in feature space F and their imbalance ratio. Through example, theoretical justifications, and empirical studies, we show ACT to be effective.

We plan to enhance ACT by closely examining the tuning of its parameters. First, when ACT decides the degree of magnification for a support vector, it needs to locate other support vectors in its neighborhood. Currently, ACT uses the margin M as the nearest-neighbor radius. We believe that the nearest neighborhood could be more adaptive to local data density. Second, Eqs. 19 and 20 set the scaling factors η_p and η_n based on the support-vector ratio. Although this rule-of-thumb has provided ACT good performance through empirical validation, we plan to investigate theoretical justification and experiment with other methods for setting η_p and η_n.

References

[1] S. Amari and S. Wu. Improving support vector machine classifiers by modifying kernel functions. *Neural Networks*, 12(6):783–789, 1999.

[2] L. Breiman. Bagging predictors. *Machine Learning*, 24:123–140, 1996.

[3] C. Burges. *Geometry and Invariance in Kernel Based Methods. In Adv. in Kernel Methods: Support Vector Learning*. MIT Press, 1999.

[4] P. Chan and S. Stolfo. Learning with non-uniform class and cost distributions: Effects and a distributed multi-classifier approach. *Workshop Notes KDD-98 Workshop on Distributed Data Mining*, pages 1–9, 1998.

[5] N. Chawla, K. Bowyer, L. Hall, and W. P. Kegelmeyer. Smote:synthetic minority over-sampling technique. *International Conference on Knowledge Based Computer Systems*, 2000.

[6] H. Cohn. *Conformal Mapping on Riemann Surfaces*. Dover Pubns, 1980.

[7] C. Cortes and V. Vapnik. Support vector networks. *Machine Learning*, 20:273–297, 1995.

[8] N. Cristianini and J. Shawe-Taylor. *An Itroduction To Support Vector Machines*. Cambridge University Press, 2000.

[9] T. Dietterich and G. Bakiri. Solving multiclass learning problems via error-correcting output codes. *Journal of Artifical Intelligence Research*, 2:263–286, 1995.

[10] T. Joachims. Text categorization with support vector machines: learning with many relevant features. *Proceedings of ECML-98, 10th European Conference on Machine Learning*, pages 137–142, 1998.

[11] M. Kubat and S. Matwin. Addressing the curse of imbalanced training sets: One-sided selection. *Proceddings of the Fourteenth International Conference on Machine Learning*, pages 179–186, 1997.

[12] Y. Lin, Y. Lee, and G. Wahba. Support vector machines for classification in nonstandard situations. *Machine Learning*, 46:191–202, 2002.

[13] J. R. Quinlan. *Programs for Machine Learning*. Morgan Kaufmann, San Mateo, CA, 1993.

[14] S. Tong and E. Chang. Support vector machine active learning for image retrieval. *Proceedings of ACM International Conference on Multimedia*, pages 107–118, October 2001.

[15] V. Vapnik. *The Nature of Statistical Learning Theory*. Springer, New York, 1995.

[16] K. Veropoulos, C. Campbell, and N. Cristianini. Controlling the sensitivity of support vector machines. *Proceedings of the International Joint Conference on Artificial Intelligence*, pages 55–60, 1999.

[17] S. Wu and S. Amari. Conformal transformation of kernel functions: A data-dependent way to improve the performance of support vector machine classifiers. *Neural Processing Letter*, 15(1), 2002.